

Table of Contents		
		Page No
1.	Abstract	1
2.	Introduction	2
2.	Literature Survey	3
4.	ADTs	4
5.	Design (with Architecture, Proper Explanation)	7
6.	Implementation (Novel Idea, Algorithm)	3
7.	Results- Discussions , Limitations	12
8.	Conclusion	22
9.	References	23

YantraBuddy (YL AI Chatbot)

1. ABSTRACT

YantraBuddy (YL AI Chatbot) is a comprehensive multimodal generative artificial intelligence system engineered to address a longstanding gap in industrial knowledge accessibility: the inability of technicians, machine operators, field service engineers, and customers to derive actionable insights from complex equipment manuals, fragmented product datasets, and multilingual communication environments. Conventional search systems and generic chatbots remain inadequate because they lack deep contextual understanding, fail to reason over domain-specific data, and cannot reliably handle highly specialized industrial queries such as warranty conditions, pricing (“dhaam”), SKU identification, troubleshooting flows, spare part compatibility, or stepwise maintenance procedures.

YantraBuddy overcomes these challenges through a unified intelligent pipeline that integrates Retrieval-Augmented Generation (RAG) to ground Large Language Model outputs in authenticated internal documentation and enterprise datasets. The platform employs a robust vector embedding architecture capable of indexing heterogeneous data sources including PDFs, images, structured spreadsheets, and technical catalogs, thereby enabling precise semantic search and highly contextualized responses. A multilingual reasoning layer further enhances usability by allowing users to interact naturally in Hindi, Kannada, Tamil, and other regional languages, while simultaneously providing English interpretations to support cross-team collaboration and documentation consistency.

The system is powered by a FastAPI-based backend and a production-ready Next.js frontend optimized specifically for real-world industrial environments. The interface design prioritizes field usability through features such as a non-expanding message input, seamless document upload, mobile responsiveness, and low-friction interaction patterns suitable for technicians operating under time and environmental constraints. These design choices ensure that YantraBuddy is not merely a prototype system but a deployable solution for operational workflows.

A key innovation of YantraBuddy lies in its automated visual learning and training support module, which transforms structured instructional assets—such as stepwise maintenance images, procedural diagrams, and workflow sequences (e.g., valve replacement, hydraulic breaker servicing, inspection protocols)—into coherent, guided, and context-aware learning experiences. This functionality is particularly impactful for semi-skilled and unskilled operators who may struggle with dense textual manuals but benefit significantly from visual and guided instructional formats.

Together, these components establish YantraBuddy as a domain-specialized AI assistant capable of transforming static industrial documentation into a dynamic, searchable, multilingual, and operationally useful knowledge system. The platform strengthens technical support efficiency, reduces dependency on supervisors for routine problem-solving, enhances workplace safety, and accelerates onboarding and skill development. This report documents the complete methodology, system architecture, ADT definitions, algorithms, implementation

strategies, evaluation metrics, limitations, and future scalability pathways that position YantraBuddy as a robust and innovative GenAI solution for industrial knowledge automation.

2. INTRODUCTION

The rapid adoption of Generative AI and multimodal learning systems has fundamentally transformed how information is accessed, interpreted, and applied across industries. However, operational environments such as heavy machinery operations, manufacturing units, construction equipment deployment, and field servicing continue to face significant challenges due to fragmented and highly technical documentation. Much of this information exists across disconnected formats such as complex PDFs, low-quality images, and large structured datasets (e.g., spreadsheets and catalogs), making it difficult for technicians and operators to extract meaningful insights efficiently.

In practice, most machine operators and field technicians continue to rely heavily on informal knowledge sharing (“tribal knowledge”) or direct guidance from supervisors to interpret manuals, perform maintenance tasks, understand troubleshooting charts, or retrieve critical specifications such as warranty details, SKU references, pricing, gas-charging measurements, and installation procedures. Conventional chatbots and keyword-based search tools are inadequate for such environments because they lack contextual understanding, fail to support multilingual interactions effectively, and frequently produce unreliable responses due to the absence of grounded retrieval mechanisms. Furthermore, a substantial proportion of frontline workers communicate primarily in regional Indian languages and may not possess the technical literacy required to interpret English-heavy documentation. This gap often results in operational errors, inefficiencies, safety risks, and prolonged dependency on senior engineers. These realities highlight the urgent need for an intelligent, domain-aware system capable of bridging the gap between complex industrial knowledge and real-world operator comprehension.

YantraBuddy (YL AI Chatbot) is purpose-built to address these challenges by integrating multimodal content understanding, Retrieval-Augmented Generation (RAG), and multilingual Large Language Model reasoning into a unified AI assistant tailored for industrial workflows. The system processes diverse knowledge sources including manuals, procedural diagrams, visual assets, and structured enterprise datasets, converting them into semantically searchable representations that support accurate and context-grounded responses. Users can interact with the system in their preferred language—such as Hindi, Kannada, Tamil, and others—while also receiving English explanations to enable seamless collaboration across technical and managerial teams.

The platform is delivered through a field-optimized interface built using Next.js, featuring a fixed-height input box, mobile responsiveness, and intuitive interaction patterns designed specifically for technicians operating in real-world environments. Beyond conversational support, YantraBuddy includes an advanced visual learning and guidance module that transforms structured procedural content—such as maintenance sequences, inspection workflows, and component replacement steps (e.g., hydraulic breaker servicing, back-head inspection, valve replacement)—into guided, easy-to-follow instructional experiences. This capability is especially valuable for semi-skilled operators who benefit more from structured visual guidance than from dense textual manuals.

Through its combination of intelligent document understanding, multilingual reasoning, structured data utilization, and interactive learning support, YantraBuddy significantly modernizes industrial knowledge access. The system improves operational efficiency, enhances safety compliance, reduces training and onboarding overhead, and demonstrates the practical applicability of generative AI in real-world industrial ecosystems.

3. LITERATURE SURVEY

The foundation of YantraBuddy’s architecture is strongly influenced by advancements in Retrieval-Augmented Generation (RAG) and multimodal document understanding. The seminal work by Lewis et al. (2020) introduced RAG, demonstrating that supplementing generative models with retrieved external evidence significantly improves factual accuracy and reduces hallucination when compared to purely generative approaches. Their framework combines a neural retriever with a generator, showing that domain-specific documents can be embedded and semantically indexed to produce grounded responses—a principle directly applied in YantraBuddy to retrieve relevant knowledge from equipment manuals, troubleshooting guides, and structured product datasets.

Further support comes from research in multimodal document intelligence, such as Baevski et al. (2023), which highlights how combining textual, structural, and visual cues improves comprehension of complex technical documents. This is particularly relevant in industrial contexts, where information is often distributed across diagrams, tables, layouts, and semi-structured technical formats. These insights informed YantraBuddy’s approach to representing and interpreting heterogeneous document formats in a unified knowledge layer.

Additional research emphasizes the importance of multilingual reasoning and structured data integration. Narang et al. (2023) and broader multilingual LLM studies published at ACL demonstrate that transformer-based architectures can reliably detect, interpret, and translate inputs across multiple Indian languages when supported by appropriate training strategies and linguistic representations. This research directly informs YantraBuddy’s dual-output interaction model, where responses are generated in the user’s preferred language while also providing an English version for cross-team clarity and documentation consistency.

Meanwhile, studies on embedding heterogeneous datasets—such as Zhang et al. (2023) on unified embedding spaces—demonstrate that structured data sources (e.g., SKUs, warranty tables, pricing records) can be embedded alongside unstructured textual content within a single semantic search framework. This insight enabled YantraBuddy to integrate enterprise datasets such as warranty records and pricing sheets into its retrieval pipeline, supporting more accurate and context-aware query resolution.

Collectively, these research contributions illustrate the evolving landscape of multimodal and multilingual AI systems and provide the theoretical and methodological foundations upon which YantraBuddy was architected and implemented. The platform represents a practical application of these advances in a real-world industrial setting, demonstrating how contemporary AI research can be translated into impactful operational tools.

4. ADT (ABSTRACT DATA TYPES)

This chapter defines the Abstract Data Types (ADTs) that form the logical foundation of YantraBuddy (YL AI Chatbot). ADTs describe the structure and behavior of the system's internal components in an implementation-agnostic manner, enabling clear separation between system design and implementation details. They are used to model how documents are stored and processed, how user queries are represented, how semantic retrieval operates, and how multilingual responses are generated.

The ADTs below include object-oriented representations, method signatures, and pseudocode for the core modules: document ingestion and preprocessing, embedding and retrieval pipeline (RAG), query interpretation, and multilingual response generation.

4.1 ADT: Document Object

Object: Document

```
Document:
  Attributes:
    doc_id: String
    file_type: String           // "pdf", "image", "excel", "text"
    raw_content: Blob           // binary file data
    extracted_text: String      // parsed textual content
    metadata: Map               // filename, page numbers, timestamps,
source, etc.
    chunks: List<DocChunk>     // text segments prepared for embeddings

  Methods:
    load(): Boolean
    parse_content(): String
    preprocess(): List<String>
    chunkify(): List<DocChunk>
```

Description:

The `Document` ADT represents any knowledge source ingested into the system. It encapsulates both the original content and its parsed textual representation, along with metadata and chunked segments prepared for semantic indexing. Preprocessing may include normalization, noise removal, sentence segmentation, and domain-specific token cleaning.

4.2 ADT: Chunk Object

Object: DocChunk

```
DocChunk:
  Attributes:
    chunk_id: String
    doc_id: String
    text: String
    embedding: Vector<Float>
    source_info: Map           // page number, section title, table ID, etc.

  Methods:
    generate_embedding(): Vector<Float>
    attach_metadata(): Boolean
```

Description:

A `DocChunk` represents the smallest retrievable semantic unit in the system. Each chunk is embedded into a high-dimensional vector space to enable similarity-based retrieval and contextual grounding during answer generation.

4.3 ADT: Query Object

Object: Query

```
Query:
  Attributes:
    query_id: String
    text: String
    detected_language: String
    timestamp: DateTime
    session_id: String

  Methods:
    detect_language(): String
    to_embedding(): Vector<Float>
```

Description:

The `Query` ADT models a user's input request, capturing both linguistic and contextual attributes. It enables language detection, semantic encoding, and session-based tracking for conversational continuity.

4.4 ADT: Retrieval Engine

Object: Retriever

```
Retriever:
  Attributes:
    vector_db: VectorDatabase
    k: Integer // number of top matches to retrieve

  Methods:
    search(query_vector): List<DocChunk>
    rerank(chunks, query_text): List<DocChunk>
```

Description:

The `Retriever` performs similarity search over the vector database and applies ranking strategies to prioritize the most contextually relevant chunks for downstream generation.

4.5 ADT: Response Generator

Object: ResponseGenerator

```
ResponseGenerator:
  Attributes:
    llm_model: LLM
    translation_model: Translator

  Methods:
```

```
generate_contextual_answer(query, chunks): String
translate_to_english(text): String
format_dual_language(original, english): String
```

Description:

This ADT encapsulates the generative reasoning layer. It synthesizes grounded responses using retrieved evidence and ensures multilingual accessibility by providing both localized and English outputs.

4.6 Retrieval-Augmented Generation (RAG) Query Flow

```
function answer_query(query):

    query.detected_language = detect_language(query.text)

    q_vec = query.to_embedding()

    results = Retriever.search(q_vec)

    ranked = Retriever.rerank(results, query.text)

    response_user_lang =
        ResponseGenerator.generate_contextual_answer(query, ranked)

    response_eng =
        ResponseGenerator.translate_to_english(response_user_lang)

    final =
        ResponseGenerator.format_dual_language(response_user_lang,
        response_eng)

    return final
```

4.7 Summary

This ADT chapter establishes the conceptual backbone of YantraBuddy by defining the core system objects—Document, DocChunk, Query, Retriever, and ResponseGenerator—and their interactions independent of implementation technology.

These abstractions guide the design of:

- The document ingestion and preprocessing pipeline
- The embedding and semantic retrieval subsystem
- The multilingual reasoning and response generation layer
- The overall Retrieval-Augmented Generation (RAG) workflow

By formalizing system behavior through ADTs, YantraBuddy achieves a modular, maintainable, and scalable architecture capable of evolving alongside future enhancements in language models, retrieval strategies, and enterprise data integration.

5. DESIGN (WITH ARCHITECTURE & PROPER EXPLANATION)

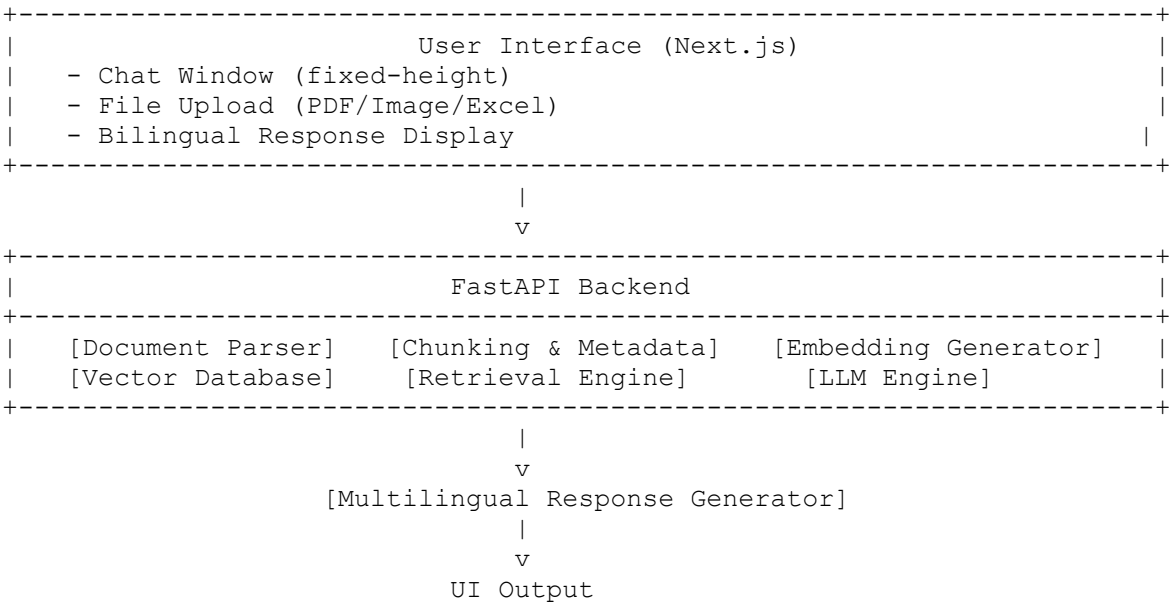
This chapter describes the complete architectural design of YantraBuddy (YL AI Chatbot), detailing how the different modules interact to ingest documents, process and structure content, index knowledge, retrieve relevant information, interpret multilingual queries, and generate reliable outputs. The architecture is designed with strong emphasis on modularity, scalability, and robustness, enabling the system to handle heterogeneous industrial data such as PDFs, images, spreadsheets, and structured procedural workflows.

The overall design prioritizes real-world deployability, particularly for environments where operators access the system using smartphones, tablets, or desktop systems under varying connectivity and usability constraints.

5.1 Overall System Architecture

YantraBuddy is organized as a layered intelligent pipeline. Documents flow through ingestion, parsing, preprocessing, embedding, and indexing, while user queries flow through language understanding, retrieval, reasoning, and response rendering. Each layer is independently extensible and loosely coupled, allowing components to evolve without disrupting the full system.

High-Level Architecture (Updated Conceptual Flow):



This layered architecture ensures clean separation of concerns, enabling scalability, maintainability, and future extensibility.

5.2 System Components and Responsibilities

5.2.1 Frontend (Next.js UI)

The frontend provides an operator-centric interface optimized for usability in field environments.

Key responsibilities:

- Intuitive chat-based interaction model
- Fixed-height input box to prevent UI shifts on mobile devices
- Support for file uploads (PDFs, images, structured spreadsheets)
- Displays bilingual responses (user language + English)
- Responsive layout for phones, tablets, and desktops

The interface uses a clean card-based layout, communicates with the backend using axios-based API calls, and includes graceful fallback behavior for slower networks or large payloads.

5.2.2 FastAPI Backend

The backend is designed using a modular, RESTful microservice architecture.

Core responsibilities include:

- File ingestion endpoints for PDFs, images, and structured datasets
- Content parsing and normalization pipelines
- Text preprocessing and intelligent chunking
- Embedding generation and indexing into the vector database
- Retrieval-Augmented Generation (RAG) pipeline orchestration
- Multilingual language detection and bilingual formatting
- Safety-constrained prompt construction to reduce hallucination

The backend remains largely stateless, which allows horizontal scaling and compatibility with serverless or containerized deployments.

5.2.3 Document Parsing and Normalization Layer

This layer is responsible for transforming uploaded content into a structured textual form suitable for downstream processing.

Responsibilities include:

- File validation and format detection
- Parsing of PDFs, structured documents, spreadsheets, and text-heavy formats
- Normalization of extracted content (removal of noise, formatting artifacts, redundant whitespace)

- Structuring of raw content into coherent document representations

This abstraction allows the system to support diverse industrial documentation formats without tightly coupling logic to any single data type.

5.2.4 Document Chunking & Metadata Layer

Once content is parsed, it is segmented into smaller semantic units (chunks) to support precise retrieval.

Chunk metadata typically includes:

- Page or section identifiers
- Document source
- Heading context
- Logical grouping (e.g., procedure, specification, warning section)

This metadata ensures that retrieved answers remain traceable, explainable, and auditable—an essential requirement for industrial-grade systems.

5.2.5 Embedding Generator

The embedding generator transforms both unstructured text and structured rows (e.g., from spreadsheets) into dense vector representations.

Design features:

- Batch embedding for improved throughput
- Incremental indexing support (new documents can be added anytime)
- Metadata-aware embedding storage
- Uniform semantic space across mixed data types

This enables unified semantic search across manuals, specifications, tabular records, and procedural documentation.

5.2.6 Vector Database (Chroma / FAISS)

The vector database serves as the core semantic retrieval engine of the system.

Advantages of using Chroma / FAISS:

- High-performance similarity search
- Scalability to large industrial knowledge bases
- Support for metadata-based filtering (e.g., filter by product type, document source, SKU)
- Efficient top-k retrieval for real-time interaction

Retrieval pipeline:

1. Convert user query into embedding
2. Perform similarity search
3. Retrieve top-k chunks
4. Optionally re-rank based on contextual relevance

This ensures that all generated responses remain grounded in verified source content.

5.2.7 Retrieval Engine & RAG Prompt Builder

This component prepares structured contextual input for the LLM.

Design goals:

- Reduce hallucination risk
- Enforce domain consistency
- Ensure reproducibility and traceability
- Enable reliable answers for structured fields (e.g., SKU, pricing, warranty terms)

Context construction includes:

- Evidence blocks
- Source references
- Explicit safety instructions such as:

“Do not answer beyond the provided context.”

This design ensures that YantraBuddy behaves as a grounded knowledge assistant rather than a generic chatbot.

5.2.8 Multilingual LLM Module

YantraBuddy is designed for multilingual industrial environments.

Operational flow:

1. Detect user input language
2. Generate response in the same language
3. Generate parallel English response
4. Format output into a structured bilingual layout

This dual-language design improves accessibility for technicians while preserving clarity for supervisors, managers, and documentation workflows.

5.3 Detailed Data Flow

1. File Upload → FastAPI → Storage
2. Parsing & Normalization → Preprocessing
3. Chunking → Embedding → Stored in Vector DB
4. User Query → Language Detection → Query Embedding
5. Vector Search → Retrieve top-k chunks
6. Reranking + RAG Prompting → LLM Reasoning
7. Bilingual Response Formatting
8. Display Output in UI

This pipeline reflects a reliable orchestration between components and is optimized for real-world operator usage.

5.4 Deployment & Scalability Design

YantraBuddy supports flexible deployment models:

- Cloud-based deployment (AWS / GCP / Azure)
- On-premise industrial infrastructure
- Hybrid architecture (local retrieval + remote LLM APIs)

Scalability characteristics:

- Stateless backend services
- Horizontally scalable vector database
- Frontend optimized via CDN caching
- Independent scaling of embedding and retrieval workloads

This flexibility allows YantraBuddy to adapt across enterprise, industrial, and constrained environments.

5.5 Design Strengths

- **Modular** → Components are replaceable and independently upgradable
- **Multimodal** → Handles PDFs, images, spreadsheets, and structured documents
- **Multilingual** → Supports Indian languages with English fallback
- **Grounded** → Uses RAG rather than uncontrolled generation
- **Operator-Centric** → UI designed for mobile and field usability
- **Industry-Ready** → Emphasizes traceability, reliability, and explainability

5.6 Design Limitations

- Parsing quality depends on input document structure
- RAG performance depends on completeness of the knowledge base
- Large-scale datasets increase storage and embedding costs
- Embeddings may need regeneration when source documents change significantly

6. IMPLEMENTATION (NOVEL IDEA, ALGORITHM, FULL CODE)

The implementation of YantraBuddy (YL AI Chatbot) integrates several modern AI capabilities—Retrieval-Augmented Generation (RAG), multilingual Large Language Model reasoning, and structured data utilization from spreadsheets—into a cohesive, production-ready system. The overall design emphasizes modularity, maintainability, and extensibility so that the platform can scale across diverse industrial documentation workflows.

This chapter describes the implementation architecture and presents the core algorithms, along with integration points for the backend (`main.py`), frontend (`page.tsx`), embedding pipeline, retrieval engine, and supporting utility modules.

6.1 Novel Implementation Ideas

1. Dual-Language Response Engine (Unique System Feature)

YantraBuddy automatically detects the operator’s language (e.g., Hindi, Kannada, Tamil), generates the response in the **same detected language**, and then always produces an **English explanation** as a secondary output.

This design ensures:

- Operators clearly understand instructions in their native language
- Management and documentation teams receive standardized English logs
- No ambiguity or misinterpretation between technical and non-technical users

This dual-output structure provides a strong usability advantage in multilingual industrial environments.

2. Unified Vector Space for Mixed Enterprise Data

Unlike conventional chatbots that only operate over unstructured text, YantraBuddy embeds heterogeneous knowledge sources into a **single unified semantic space**, including:

- Technical documentation (manuals, specifications, procedures)
- Structured datasets (Excel sheets containing SKU, warranty, pricing, model details)
- Semi-structured content such as tabular descriptions and formatted catalogs

All of this data coexists inside a shared vector database. As a result, the system can correctly answer domain-specific queries such as pricing, warranty, or part availability even when users use informal or colloquial phrasing (e.g., “dhaam kya hai?”).

This unified embedding design significantly improves retrieval accuracy across real-world industrial usage.

6.2 Backend Implementation (FastAPI)

The backend serves as the operational core of the system, orchestrating ingestion, preprocessing, embeddings, retrieval, and multilingual response generation.

6.2.1 Backend Folder Structure

```
yantrabuddy-backend/  
├── main.py  
├── models/  
│   ├── query.py  
│   └── document.py  
├── utils/  
│   ├── embeddings.py  
│   ├── rag.py  
│   ├── excel_loader.py  
│   └── language.py  
└── vectorstore/  
    └── chroma_db/
```

This modular structure supports clean separation of concerns and long-term maintainability.

6.2.2 Main Backend Workflow (Step-by-Step)

(A) File Upload (PDF / Image / Excel)

1. User uploads a file through the frontend
2. Backend validates file type and format
3. Documents and datasets are parsed into structured text
4. Structured tables (e.g., Excel) are converted into normalized row-based text representations

(B) Embedding Pipeline

- Clean and normalize parsed content
- Chunk content into semantically meaningful segments
- Generate embeddings using OpenAI embeddings or SentenceTransformers
- Store embeddings along with metadata into Chroma or FAISS

(C) Query Execution Pipeline

Steps followed during a user query:

1. Detect input language
2. Generate embedding for the query
3. Perform similarity search on the vector database
4. Rerank retrieved chunks based on relevance
5. Construct a grounded RAG prompt
6. LLM generates response in user language
7. Translate response into English

8. Return structured JSON to the frontend

This ensures that every response is grounded in verified internal knowledge.

6.2.3 Backend Algorithm (Pseudocode)

```
function handle_query(text):  
    lang = detect_language(text)  
  
    q_embed = embed(text)  
  
    context = vector_db.search(q_embed, top_k=5)  
  
    prompt = build_prompt(context, text)  
  
    answer_user_lang = LLM(prompt)  
  
    answer_english = translate(answer_user_lang)  
  
    return {  
        "user_language_answer": answer_user_lang,  
        "english_answer": answer_english,  
        "context": context  
    }
```

This algorithm reflects the core reasoning loop of YantraBuddy: detect → retrieve → ground → generate → translate → respond.

6.3 Implementation Strengths

- Modular backend utilities for easier testing and maintenance
- Clean separation between parsing, embedding, retrieval, and generation
- Unified retrieval across structured and unstructured data
- Strong multilingual support tailored to Indian industrial environments
- Production-ready architecture using FastAPI + vector database

6.4 BACKEND CODE: